강화학습 알고리즘 문헌 정리

TRPO, PPO, SAC, GRPO, DAPO, SDPO와 다중 에이전트 · 보상 설계 문헌

CS30707 팀 프로젝트 참고 문서

2026년 10월 30일

1 문서의 목적과 읽기 범위

이 문서는 3대3 territory-control 게임에서 *팀원 간 정보 구조*(독립 결정, 순차 결정, 의도 공유)와 *보상 목표*(팀 승리, 팀 승리와 개인 생존)가 협력 행동에 미치는 영향을 분석하기 위해 참고할 강화학습 알고리즘을 정리한다. 알고리즘 자체의 목적함수, 학습 신호, 특징 및 한계를 중심으로 기술한다. 새로운 알고리즘 제안이나 코드 수준의 구현 설계는 범위에 포함하지 않으며, 프로젝트와 관련된 설계 판단은 §11에 개념 수준으로만 정리한다.

지정 자료의 범위는 다음과 같다.

1. **OpenAI Spinning Up**: `https://spinningup.openai.com/en/latest/` 중 TRPO, PPO, SAC만 검토한다. 각 알고리즘의 개별 문서 링크는 해당 절에 수록한다.
2. **DeepSeek-R1**: `https://arxiv.org/pdf/2501.12948` 중 Section 2의 GRPO 관련 설명과 Appendix A.3의 PPO–GRPO 비교를 중심으로 정리한다. Section 2.2의 보상 설계는 알고리즘 이해에 필요한 수준으로만 포함한다.
3. **DAPO**: `https://dapo-sia.github.io/` 및 기술 보고서 `https://arxiv.org/pdf/2503.14476`를 바탕으로 주요 설계 변경과 실험적 관찰을 정리한다.
4. **SDPO**: `https://self-distillation.github.io/SDPO`를 바탕으로 연구 아이디어만 간략히 정리한다. 프로젝트에서 사용할지는 미정이다.

지정 자료만으로는 실제로 사용할 다중 에이전트 PPO, 순차 결정과 통신, 보상 shaping을 설명할 수 없다. 따라서 다음을 **추가 문헌**으로 포함한다.

5. 각 알고리즘의 원 논문과 PPO 구현 세부(§3–§5).
6. 다중 에이전트 강화학습: parameter sharing, IPPO/MAPPO, credit assignment, 순차 결정, learned communication(§9).
7. 보상 설계: potential-based reward shaping과 시간 제한 처리(§10).

지정 자료와 추가 문헌을 읽는 우선순위는 §13에 정리한다.

프로젝트 저장소는 `https://github.com/a11cho/2026F_CS30707_Team_1`이다 [7]. 2026년 10월 30일 기준으로 저장소에는 게임 규칙 문서(`env.md`)와 환경 구현 초안(`territory_rl/`)이 있다. 본 문서의 프로젝트 서술은 저장소의 맵 · 규칙과 팀이 정한 6개 실험 조건(§11)을 따른다. 아직 확정되지 않은 사항은 본문에 [**미정**]으로 표시하며, 실험 계획이 바뀌면 문서를 갱신한다.

2 공통 개념 및 표기

Policy $\pi_\theta(a \mid s)$는 state s에서 행동 a를 선택할 확률을 나타낸다. θ_{old}는 데이터를 수집한 이전 policy의 파라미터이며, 이 policy를 $\pi_{\text{old}} := \pi_{\theta_{\text{old}}}$로 줄여 쓴다. 다음의 importance ratio는 policy 변화의 정도를 표현한다.

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}. \tag{1}$$

State-value function $V^\pi(s)$, action-value function $Q^\pi(s,a)$, advantage $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$를 사용한다. Advantage는 특정 행동이 해당 상태의 평균적인 선택보다 얼마나 유리한지를 뜻한다.

Discounting factor $\gamma \in [0,1)$은 미래 보상의 가중치를 정한다. k틱 뒤 보상의 가중치는 γ^k이므로 유효 지평은 대략 $1/(1-\gamma)$틱이다. 예를 들어 $\gamma = 0.99$이면 약 100틱이고, 256틱 뒤 보상의 가중치는 $0.99^{256} \approx 0.08$이다.

GAE. PPO와 TRPO에서 자주 사용하는 generalized advantage estimation(GAE) [10]은 TD residual

$$\delta_t = r_t + \gamma(1 - d_{t+1})V(s_{t+1}) - V(s_t) \tag{2}$$

를 이용해

$$\widehat{A}_t^{\text{GAE}(\gamma,\lambda)} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l} \tag{3}$$

형태로 계산한다. d_{t+1}은 s_{t+1}이 종료 상태이면 1, 아니면 0이다. $\lambda \in [0,1]$는 bias–variance trade-off를 조절한다. $\lambda = 0$이면 한 단계 TD 오차 δ_t만 쓰므로 분산은 작지만 V가 부정확한 만큼 bias가 생긴다. $\lambda = 1$이면 Monte Carlo return에서 $V(s_t)$를 뺀 값이 되어 V로 인한 bias는 없지만 분산이 크다. 이 구조는 TD(λ)의 λ-return과 같다(Spinning Up의 GAE 설명 참고). Value function의 학습 목표(return target)로는 보통 $\widehat{R}_t = \widehat{A}_t + V(s_t)$를 쓰며, Spinning Up 구현은 reward-to-go $\sum_{l \geq 0} \gamma^l r_{t+l}$를 쓴다.

반면 GRPO와 DAPO의 *group-relative advantage*는 value function을 학습하지 않고 같은 입력에 대한 출력 집단의 보상을 정규화한다. 이 둘은 이름이 advantage로 같아도 추정 방식과 적용 단위가 다르다.

종료와 중단. 에피소드가 끝나는 이유는 두 가지로 구분해야 한다 [38]. 2026.10.30. 팀미팅 이후 정한 종료(승패가 정해지는 마지막 틱 등)는 *termination*으로 보고 $d = 1$, 즉 이후 가치를 0으로 둔다. 학습 편의를 위해 rollout을 중간에 자른 경우는 *truncation*이므로 마지막 상태의 V로 bootstrap해야 한다. 시간 제한이 과제의 일부라면 남은 시간을 관측에 넣어야 한다. 그렇지 않으면 같은 관측이라도 남은 시간에 따라 가치가 달라져, 가치함수가 이를 구별할 수 없다. 이 프로젝트의 종료 틱 T는 승패가 정해지는 과제 내 종료에 해당한다.

3 Trust Region Policy Optimization (TRPO)

출처: `https://spinningup.openai.com/en/latest/algorithms/trpo.html` [1]; 원 논문 [8].

3.1 문제의식과 목적함수

정책 경사(policy gradient) 방법에서는 파라미터 공간에서의 작은 이동이 정책, 즉 행동 분포의 작은 변화를 보장하지 않는다. 그래서 한 번의 큰 업데이트로 성능이 급격히 무너질 수 있다. TRPO

는 정책을 개선하되 이전 정책과의 평균 KL divergence를 지정된 범위 안에 유지하도록 제약한다.

$$\max_{\theta} \quad \mathcal{L}(\theta_{\text{old}}, \theta) = \mathbb{E}_{s,a\sim\pi_{\text{old}}} \left[\frac{\pi_\theta(a \mid s)}{\pi_{\text{old}}(a \mid s)} A^{\pi_{\text{old}}}(s,a) \right], \tag{4}$$

$$\text{s.t.} \quad \overline{D_{\text{KL}}}(\pi_\theta \| \pi_{\text{old}}) \le \delta. \tag{5}$$

여기서 $\overline{D_{\text{KL}}}$은 이전 정책이 방문한 상태에 대한 평균 KL divergence이고, δ는 허용하는 policy 변화량이다. 원 논문 [8]은 surrogate $\mathcal{L}$에서 최대 KL에 비례하는 항을 뺀 값이 실제 성능의 하한이 된다는 단조 개선 이론에서 출발하고, 실용 알고리즘에서는 이를 평균 KL 제약으로 완화한다.

3.2 실제 업데이트

현재 파라미터 근방에서 목적함수(objective)를 일차, KL constraint를 이차로 근사한다. $d = \theta - \theta_{\text{old}}$, $g = \nabla_\theta \mathcal{L}$, H를 평균 KL의 Hessian이라 하면,

$$\max_{d} \; g^\top d \quad \text{s.t.} \quad \frac{1}{2} d^\top H d \le \delta \qquad \Longrightarrow \qquad d_* = \sqrt{\frac{2\delta}{g^\top H^{-1} g}}\, H^{-1} g. \tag{6}$$

실제 업데이트는 $\theta_{\text{new}} = \theta_{\text{old}} + \alpha^j d_*$이다. $\alpha \in (0,1)$은 backtracking 계수이고, j는 실제 KL 제약과 surrogate 개선을 모두 만족하는 가장 작은 음이 아닌 정수다. 이 과정에는 세 가지 계산 기법이 쓰인다.

- **Hessian–vector product**: H를 행렬로 만들지 않고 Hx만 계산한다. 평균 KL의 gradient와 x의 내적을 한 번 더 미분하면 Hx가 나온다.
- **Conjugate gradient**: $Hx = g$를 Hx 곱셈만 반복해서 근사적으로 푸는 방법이다. H^{-1}을 직접 구하지 않으며, Spinning Up 구현은 10회 반복한다.
- **Backtracking line search**: 이차 근사는 step이 클수록 부정확하므로, step을 α^j배로 줄여 가며 실제 KL과 surrogate 개선을 확인한다.

Spinning Up의 TRPO도 PPO처럼 별도의 value functino을 학습하고 GAE로 advantage를 추정한다.

3.3 해석과 한계

TRPO는 *on-policy*이며 이산 및 연속 행동을 모두 다룰 수 있다. 정책 업데이트 안정성을 이해하기 위한 출발점으로 중요하지만, Hessian–vector product, conjugate gradient 및 line search가 필요하므로 구현 복잡도가 높다. 이후 PPO의 간단한 제한적 업데이트를 이해하기 위한 이론적 비교 대상이다.

4 Proximal Policy Optimization (PPO)

출처: `https://spinningup.openai.com/en/latest/algorithms/ppo.html` [2]; 원 논문 [9]; 구현 세부 [11, 12].

4.1 핵심 아이디어

PPO는 TRPO와 같은 문제, 즉 지나치게 큰 policy 업데이트를 다루지만, 이차 근사와 conjugate gradient 없이 일차 최적화만으로 계산 효율을 높인다. 원 논문은 KL penalty 형태와 clipping 형태를 모두 제안하며, Spinning Up은 주로 **PPO-Clip**을 설명한다. PPO-Clip은 다음의 surrogate

objective를 최대화한다.

$$L^{\text{clip}}(\theta) = \mathbb{E}_t\Big[\min\Big(\rho_t(\theta)\widehat{A}_t,\, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\widehat{A}_t\Big)\Big]. \tag{7}$$

$\widehat{A}_t > 0$이면 유리한 행동의 확률을 높이는 방향으로 학습하지만 $1+\epsilon$을 넘는 확률비에서 추가 개선의 이득을 제한한다. $\widehat{A}_t < 0$이면 불리한 행동의 확률을 줄이는 효과가 $1-\epsilon$ 아래에서 제한된다. Clipping은 *엄밀한 KL 제약*이 아니며, 실제 정책 간 KL을 항상 일정 값 이하로 보장하지는 않는다.

KL penalty 형태는 $\mathbb{E}_t[\rho_t\widehat{A}_t - \beta\, D_{\text{KL}}(\pi_{\text{old}}(\cdot \mid s_t) \| \pi_\theta(\cdot \mid s_t))]$를 최대화하고, 관측된 KL이 목표보다 크면 β를 키우고 작으면 줄인다 [9]. 원 논문의 실험에서는 clipping 형태가 더 좋은 성능을 보였다.

4.2 전체 손실

원 논문은 정책과 가치함수가 파라미터를 공유하는 경우 다음의 결합 목적함수를 최대화한다 [9].

$$L(\theta) = \mathbb{E}_t\Big[\min\Big(\rho_t\widehat{A}_t,\, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\widehat{A}_t\Big) - c_1\big(V_\theta(s_t) - \widehat{R}_t\big)^2 + c_2\,\mathcal{H}\big(\pi_\theta(\cdot \mid s_t)\big)\Big]. \tag{8}$$

둘째 항은 value function(*critic*) 손실이고, 셋째 항은 entropy bonus로 정책이 너무 일찍 결정적으로 굳는 것을 막아 탐색을 유지한다. Spinning Up 구현은 actor와 critic을 별도 네트워크로 두고 entropy 항을 쓰지 않는다. 반면 원 논문의 Atari 설정을 비롯한 많은 이산 행동 구현은 entropy 항을 사용한다 [9, 11].

4.3 학습 흐름

1. 현재 정책 π_{old}로 여러 환경에서 일정 길이의 rollout을 수집한다. 각 transition에 s_t, a_t, r_t, d_t, $\log \pi_{\text{old}}(a_t \mid s_t)$, $V(s_t)$와 (사용한다면) action mask를 저장한다.
2. GAE로 $\widehat{A}_t$와 $\widehat{R}_t = \widehat{A}_t + V(s_t)$를 계산한다.
3. 같은 batch를 K epoch 동안 minibatch로 나누어 위 손실을 최적화한다. 같은 데이터를 여러 번 쓰지만 clipping이 정책 변화를 제한한다.
4. (선택) 정책 간 근사 KL이 목표값을 넘으면 그 batch에 대한 업데이트를 조기 종료한다. Spinning Up 구현이 이 방식을 쓴다.
5. 갱신된 정책으로 1로 돌아간다. 이전 batch는 버린다(*on-policy*).

원 논문의 MuJoCo 설정은 rollout 길이 2048, $\epsilon = 0.2$, $\gamma = 0.99$, $\lambda = 0.95$, epoch 10, minibatch 64, 학습률 3×10^{-4}이다 [9]. 이는 출발점일 뿐 현재 환경의 최적 값은 아니다.

4.4 구현 세부와 action masking

PPO의 성능은 목적함수만큼 구현 세부에 좌우된다 [11, 12]. 이 프로젝트와 관련이 큰 항목은 다음과 같다.

- **Advantage 정규화**: minibatch 안에서 $\widehat{A}$를 평균 0, 표준편차 1로 맞춘다. 실험 조건마다 보상 크기가 달라도 정책 gradient의 크기가 비슷하게 유지된다.
- **관측 · 보상 정규화와 gradient clipping**: 입력과 return의 크기를 일정하게 유지하고, gradient norm을 제한해 한 번의 큰 업데이트를 막는다.
- **Value clipping**: 일부 구현은 가치함수 업데이트도 clipping하지만, 대규모 비교 연구 [12]에서는 성능을 개선하지 못하거나 오히려 해칠 수 있다고 보고되었다.

- **Invalid action masking**: 선택할 수 없는 행동(이 게임에서는 cooldown 중인 공격, 맵 밖으로의 이동 등)의 logit을 매우 작은 값으로 바꾼 뒤 softmax를 취한다. rollout 때와 업데이트 때 *같은* $mask$로 $\log \pi$를 계산해야 importance ratio가 올바르다 [13].

4.5 해석과 한계

TRPO와 마찬가지로 이산 · 연속 행동 모두에 적용할 수 있으며, 일차 최적화만으로 구현할 수 있어 훨씬 간단하다. 다만 on-policy이므로 데이터 수집 비용이 크고, 보상 설계, 가치함수 및 advantage 추정 방식, 위의 구현 세부가 결과에 크게 영향을 준다.

5 Soft Actor-Critic (SAC)

출처: `https://spinningup.openai.com/en/latest/algorithms/sac.html` [3]; 원 논문 [14, 15].

5.1 최대 엔트로피 강화학습

SAC는 off-policy actor–critic으로, 기대 수익뿐 아니라 policy 엔트로피도 함께 최대화한다.

$$J(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t\geq 0}\gamma^t\left(r_t + \alpha\mathcal{H}(\pi(\cdot\mid s_t))\right)\right], \qquad \mathcal{H}(\pi(\cdot\mid s)) = -\mathbb{E}_{a\sim\pi}\log\pi(a\mid s). \tag{9}$$

α는 탐색 다양성과 기대 보상 사이의 균형을 조절한다. 정책이 너무 일찍 하나의 행동으로 굳어 나쁜 국소 최적에 수렴하는 현상을 줄이려는 접근이다.

5.2 Q-learning과 policy 업데이트

Replay buffer $\mathcal{D}$에서 transition을 샘플링하고, 두 Q function을 사용하는 clipped double-Q target을 적용한다.

$$y = r + \gamma(1-d)\left[\min_{j=1,2} Q_{\bar{\phi}_j}(s', \tilde{a}') - \alpha\log\pi_\theta(\tilde{a}'\mid s')\right], \tag{10}$$

$$\tilde{a}' \sim \pi_\theta(\cdot\mid s'), \tag{11}$$

$$L_{Q_j} = \mathbb{E}_{(s,a,r,s',d)\sim\mathcal{D}}\left[\left(Q_{\phi_j}(s,a) - y\right)^2\right]. \tag{12}$$

Policy는 buffer의 상태 $s\sim\mathcal{D}$에서 *현재 policy로 새로 샘플링한* 행동을 사용해 다음을 최대화한다.

$$\mathbb{E}_{s\sim\mathcal{D},\ \xi\sim\mathcal{N}(0,I)}\Big[\min_{j=1,2} Q_{\phi_j}\big(s, \tilde{a}_\theta(s,\xi)\big) - \alpha\log\pi_\theta\big(\tilde{a}_\theta(s,\xi)\mid s\big)\Big], \tag{13}$$

$$\tilde{a}_\theta(s,\xi) = \tanh\big(\mu_\theta(s) + \sigma_\theta(s)\odot\xi\big). \tag{14}$$

Buffer에 저장된 행동 a가 아니라 $\tilde{a}_\theta$를 쓰는 것이 핵심이다. Reparameterization 덕분에 Q를 거쳐 정책 파라미터까지 gradient가 전달된다. Spinning Up이 설명하는 최신 형태는 별도 state-value function 없이 두 Q 함수와, polyak 평균으로 갱신하는 target network를 이용한다. Spinning Up 구현은 α를 고정하지만, 후속 논문 [15]은 목표 entropy를 정해 α를 자동으로 조정하는 방법을 제안한다.

5.3 적용 조건

SAC의 장점은 off-policy 재사용과 명시적 엔트로피 보너스다. 다만 **Spinning Up의 SAC 구현은 continuous action space용**이며 tanh-squashed Gaussian policy(쉽게 말해 그냥 Gaussian policy)를 전제로 한다. 이산 행동을 사용하는 이 게임에는 이 구현을 그대로 가져올 수 없다. 정책이 행동별 확률을 직접 출력하고, 기댓값을 행동에 대해 정확히 합산하는 discrete SAC 변형 [16]이 필요하다. 따라서 본 문서에서는 PPO의 직접 대안이라기보다 탐색과 replay를 이해하기 위한 비교 대상으로 놓는다.

6 Group Relative Policy Optimization (GRPO)

출처: DeepSeek-R1, Section 2.1–2.2 및 Appendix A.3, `https://arxiv.org/pdf/2501.12948` [4]; 최초 제안은 DeepSeekMath [17].

6.1 PPO와의 차이

GRPO는 DeepSeekMath [17]에서 수학 추론 학습을 위해 처음 제안되었고, DeepSeek-R1이 이를 대규모로 사용했다. DeepSeek-R1에서의 GRPO는 같은 질문 q에 대해 이전 정책으로 여러 개의 답변 $o_1, \ldots, o_G$을 생성하고, 답변에 부여된 보상의 상대적 크기로 advantage를 구성한다. PPO의 학습된 value model/GAE 대신, 같은 질문에서 나온 *여러 rollout의 비교*를 사용한다.

$$\overline{r}_q = \frac{1}{G}\sum_{i=1}^{G} r_i, \qquad \widehat{A}_i = \frac{r_i - \overline{r}_q}{\mathrm{std}(r_1, \ldots, r_G)}. \tag{15}$$

모든 응답의 보상이 같으면 분자 $r_i - \overline{r}_q$가 0이 되어 advantage가 모두 0이 된다(구현에서는 분모에 작은 상수를 더해 0으로 나누는 것을 막는다). 그러면 그 질문은 KL 항 외에는 학습 신호를 주지 않는다. 해당 문제는 정답/오답처럼 보상이 이진일 때 자주 생기며, 이 문제는 DAPO의 dynamic sampling(§7)에서 다시 등장한다.

6.2 목적함수와 KL 항

DeepSeek-R1의 Section 2.1은 개별 답변 o_i에 대한 정책 확률비 $\rho_i = \pi_\theta(o_i \mid q)/\pi_{\mathrm{old}}(o_i \mid q)$와 group advantage를 사용해 아래 형태를 제시한다.

$$J_{\mathrm{GRPO}}(\theta) = \mathbb{E}_{q,\{o_i\}}\left[\frac{1}{G}\sum_{i=1}^{G}\left(\min\Big(\rho_i\widehat{A}_i, \mathrm{clip}(\rho_i, 1-\epsilon, 1+\epsilon)\widehat{A}_i\Big) - \beta\widehat{D_{\mathrm{KL}}}(\pi_\theta\|\pi_{\mathrm{ref}})\right)\right]. \tag{16}$$

본문에 사용된 KL 추정식은 해당 출력에 대해 다음과 같다.

$$\widehat{D_{\mathrm{KL}}} = \frac{\pi_{\mathrm{ref}}(o_i \mid q)}{\pi_\theta(o_i \mid q)} - \log\frac{\pi_{\mathrm{ref}}(o_i \mid q)}{\pi_\theta(o_i \mid q)} - 1. \tag{17}$$

이 식은 Schulman이 정리한 k3 추정량 [19]으로, 항상 0 이상이며 o_i를 π_θ에서 샘플링할 때 $D_{\mathrm{KL}}(\pi_\theta\|\pi_{\mathrm{ref}})$의 불편 추정량이다. π_{ref}는 학습 중인 정책을 기준 정책과 비교하는 데 쓰인다. *이 항은 Spinning Up에서 다루는 표준 PPO-Clip의 필수 구성요소가 아니다.* Appendix A.3은 특히 LLM RLHF 맥락의 PPO에서 토큰별 KL을 reward에 반영하는 방식과 GRPO에서 KL을 loss에 직접 넣는 방식을 비교한다.

6.3 응답 단위 표기와 토큰 단위 구현

위 식은 R1 논문의 표기를 따라 응답 전체의 확률비 ρ_i로 적었다. GRPO를 처음 정의한 DeepSeek-Math [17]와 일반적인 구현은 토큰 단위로 계산한다. 응답 o_i의 t번째 토큰에 대한 확률비를 $\rho_{i,t}$라

하면, 각 응답 안에서 토큰 손실을 평균한 뒤 응답들에 대해 평균한다.

$$J(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\Big(\min\big(\rho_{i,t}\widehat{A}_i, \mathrm{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)\widehat{A}_i\big) - \beta\,\widehat{D_{\mathrm{KL}}}_{i,t}\Big)\right]. \tag{18}$$

결과(outcome) 보상만 있으면 한 응답의 모든 토큰이 같은 advantage $\widehat{A}_i$를 갖는다. DAPO가 비교 대상으로 삼는 "원래 GRPO"의 손실 집계가 이 형태다(§7의 L_{sample}).

6.4 GRPO의 정규화가 만드는 편향

Dr. GRPO 논문 [18]은 GRPO의 두 정규화가 편향을 만든다고 지적한다. 첫째, 표준편차로 나누면 보상이 거의 같은(너무 쉽거나 너무 어려운) 질문의 advantage가 커져 그런 질문의 가중치가 과도해진다. 둘째, 응답 길이 $|o_i|$로 나누면 틀린 긴 응답의 토큰 하나하나가 받는 벌점이 작아져, 오답 응답이 길어지는 경향이 생긴다. 이 논문은 두 정규화를 제거한 변형을 제안한다. 길이 정규화 문제는 DAPO의 token-level loss가 다루는 문제를 다른 각도에서 본 것이다.

6.5 Section 2의 보상 설계와 학습 맥락

DeepSeek-R1-Zero는 supervised fine-tuning 없이 RL로 출발하며, 정답 검증이 가능한 수학 · 코딩 · 논리 문제에 대해 규칙 기반 *accuracy reward* 및 *format reward*를 사용한다. 학습 과정에서 답변 길이 및 추론 행동이 변화했다는 관찰은 알고리즘이 사용된 환경을 설명하지만, 그 자체가 GRPO의 일반적인 성능 보장을 뜻하지는 않는다.

6.6 Appendix A.3: GRPO–PPO 비교

Appendix A.3이 제시하는 핵심 비교는 (i) learned value model 제거에 따른 메모리 · 연산 비용 감소, (ii) terminal reward와 긴 추론 과정에서의 value prediction 난점, (iii) KL 항을 반영하는 위치의 차이다. 해당 부록의 실험에서 PPO의 GAE λ를 잘 조정하면 GRPO와의 격차가 줄어든다는 점도 중요하다. 즉, **GRPO가 모든 문제에서 PPO보다 우월하다는 주장은 아니다.**

6.7 3인 팀과 "group"의 구별

GRPO에서의 *group*은 **하나의 입력 조건에서 샘플링한 여러 개의 대안 출력**을 뜻한다. 게임 안에서 공동 행동하는 *3명의 팀원*과는 다른 개념이다. 다중 에이전트에 적용하려면 무엇을 동일 조건의 대안 rollout으로 정의할지 추가 모델링이 필요하므로, 3인 팀을 곧바로 GRPO의 $G = 3$으로 대응시키면 안 된다. 이 프로젝트는 자체 시뮬레이터를 쓰므로 같은 seed와 같은 시작 상태에서 에피소드를 여러 번 반복하는 것은 가능하며, 이를 group으로 정의하는 것이 자연스럽다. 다만 이 방법은 에피소드 전체를 하나의 결과로 평가하므로, 256틱 동안의 개별 행동에 대한 credit assignment를 critic 없이 해결해 주지는 않는다.

7 DAPO: Decoupled Clip and Dynamic Sampling Policy Optimization

출처: 프로젝트 페이지 `https://dapo-sia.github.io/`, 기술 보고서 `https://arxiv.org/pdf/2503.14476` [5].

7.1 등장 배경

DAPO는 긴 chain-of-thought를 생성하는 LLM의 RL 학습에서 초기 GRPO 실험이 나타낸 *entropy collapse*, 효율이 낮은 샘플, 응답 길이 편향 및 잘못된 종료 처리에 주목한다. 단일한 새로운 학습

원리라기보다 GRPO 계열 업데이트에 네 가지 실용적 변경을 결합한다. 이 논문은 기존 GRPO에서 참조 정책과의 KL penalty를 제거한 설정도 채택한다.

7.2 비대칭 clipping

PPO/GRPO의 대칭 clipping $[1-\epsilon, 1+\epsilon]$을

$$\text{clip}(\rho_{i,t},\, 1-\epsilon_{\text{low}},\, 1+\epsilon_{\text{high}}) \tag{19}$$

로 바꾸어 위쪽 허용 폭을 더 크게 설정한다. 논문은 확률이 낮은 탐색 토큰의 확률이 커지는 것이 지나치게 억제될 수 있다고 지적한다. 예를 들어 $\epsilon = 0.2$이면 $\pi_{\text{old}} = 0.01$인 토큰은 한 번의 업데이트로 최대 0.012까지만 커질 수 있지만, $\pi_{\text{old}} = 0.9$인 토큰에는 상한 1.08이 사실상 제약이 되지 않는다. 즉 대칭 clipping은 이미 확률이 높은 토큰보다 확률이 낮은 탐색 토큰을 더 강하게 묶는다. 상한을 높이면 다양한 응답을 탐색할 여지를 늘릴 수 있다. 보고된 대표 설정은 $\epsilon_{\text{low}} = 0.20$, $\epsilon_{\text{high}} = 0.28$이다. 이는 모든 환경에서의 최적 값이 아니라 논문의 실험 설정이다.

7.3 Dynamic Sampling: 유효한 집단 확보

같은 질문에 대한 G개 응답이 전부 정답이거나 전부 오답이면 보상 기반 group advantage의 정보량이 사라진다. DAPO는 이러한 질문 집단을 제외하고, 정답 · 오답이 섞여 있는 집단을 필요한 batch 크기가 될 때까지 추가 샘플링한다. 규칙 검증 $\text{is_equivalent}(a, o_i)$를 기준으로

$$0 < \sum_{i=1}^{G} \mathbb{I}\{\text{is_equivalent}(a, o_i)\} < G \tag{20}$$

를 만족하는 집단을 학습에 사용하는 것이다. 추가 생성 비용이 들지만, 학습에 유효한 gradient를 갖는 샘플의 비율을 높이려는 목적이다.

7.4 Token-level Policy Gradient Loss

원래 GRPO의 손실 집계(§6)는 각 응답 안에서 token loss를 먼저 평균내고, 다음에 응답별 loss를 평균낸다. 그 결과 길이가 긴 응답 속의 각 토큰은 상대적으로 작은 가중치를 가진다. DAPO는 응답별 길이로 먼저 나누는 대신 전체 batch에 포함된 토큰 수로 나누어 토큰을 동등하게 반영한다.

토큰별 확률비와 clipped 기여도를

$$\rho_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\text{old}}(o_{i,t} \mid q, o_{i,<t})}, \tag{21}$$

$$\ell_{i,t}(\theta) = \min\left\{\rho_{i,t}\widehat{A}_i,\, \text{clip}(\rho_{i,t}, 1-\epsilon_{\text{low}}, 1+\epsilon_{\text{high}})\widehat{A}_i\right\} \tag{22}$$

라고 하면, 집계 방식의 차이는 다음과 같다.

$$L_{\text{sample}} = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\ell_{i,t}, \tag{23}$$

$$L_{\text{token}} = \frac{1}{\sum_{i=1}^{G}|o_i|}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\ell_{i,t}. \tag{24}$$

긴 응답을 무조건 선호하는 목적은 아니다. 저품질 반복 표현에 대해서도 토큰 수준의 학습 신호가 충분히 적용되도록 하여, 응답 길이와 entropy의 불안정한 증가를 완화하려는 설계다 (의도는 그렇다는데 잘 모르겠다...).

7.5 Overlong Reward Shaping

길이 제한으로 응답이 잘린 경우 무조건 틀렸다고 벌점을 주면, 원래 좋은 추론 중이었던 출력에도 부정확한 reward가 할당될 수 있다. DAPO는 *Overlong Filtering*(잘린 샘플의 loss masking)을 살펴보고, 경계 구간에서 점진적으로 패널티를 주는 *Soft Overlong Punishment*도 제안한다.

$$R_{\text{length}}(y) = \begin{cases} 0, & |y| \le L_{\max} - L_{\text{cache}}, \\ \dfrac{L_{\max} - L_{\text{cache}} - |y|}{L_{\text{cache}}}, & L_{\max} - L_{\text{cache}} < |y| \le L_{\max}, \\ -1, & |y| > L_{\max}. \end{cases} \tag{25}$$

이 추가 길이 보상은 정답 판정 보상에 더해진다. 효과와 필요성은 긴 *생성 시퀀스*가 존재한다는 가정에 강하게 의존한다. 강화학습 일반의 관점에서는, 시간 제한으로 잘린 trajectory를 실패로 처리할지의 문제(§2의 종료와 중단)와 같은 구조다.

7.6 실험 결과와 해석 범위

논문의 Qwen2.5-32B / AIME 2024 실험에서 naive GRPO의 avg@32 정확도는 30, 전체 DAPO 구성은 50으로 보고된다. 단계별 적용 결과는 overlong filtering 36, clip-higher 38, soft overlong punishment 41, token-level loss 42, dynamic sampling 50으로 제시된다. 이 수치는 *특정 LLM · 데이터셋 · 실험 순서의 관찰*이며, 각 구성요소가 모든 환경에서 동일한 절대 성능 상승을 낸다는 의미는 아니다.

8 SDPO: Self-Distillation Policy Optimization (후보)

출처: `https://self-distillation.github.io/SDPO`, 논문 `https://arxiv.org/abs/2601.20802` [6].

GRPO와 같은 scalar outcome reward 중심의 방법은 실패의 *이유*를 충분히 활용하지 못할 수 있다. SDPO는 오류 메시지나 평가 피드백과 같은 *rich feedback*을 조건으로 제공받은 현재 모델을 *self-teacher*로 사용하고, 그 모델의 다음 토큰 분포를 원래 정책에 distillation한다. 외부 teacher나 명시적 reward model 없이 feedback을 보다 세밀한 학습 신호로 바꾸는 발상이다 (teacher, student를 모른다면 간단하게 지피티나 클로드에 질문한다).

이 방법의 학습 단위는 텍스트 토큰과 피드백 조건부 분포다. 따라서 보상만 주어지는 작은 이산 행동 게임에 SDPO를 직접 적용할 수 있다고 전제해서는 안 된다. 현재 문헌 목록에서는 **연구 방향을 이해하기 위한 보류 후보**로만 둔다.

9 다중 에이전트 강화학습

출처: 추가 문헌 [20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34].

9.1 문제 설정

에이전트 $i = 1, \ldots, N$은 전체 상태 s 대신 부분 관측 o^i를 받고, 모든 에이전트의 행동 $\boldsymbol{a} = (a^1, \ldots, a^N)$이 함께 다음 상태를 정한다. 보상을 팀이 공유하는 이런 문제를 Dec-POMDP라 한다. 단일 에이전트와 비교해 다음 세 가지가 새로 생긴다.

- **비정상성(non-stationarity)**: 에이전트 i의 입장에서는 팀원과 상대의 정책도 환경의 일부인데, 학습 중에 이들이 바뀌므로 전이 확률이 계속 변한다.

- **Credit assignment**: 팀 보상을 모두가 똑같이 받으면 각 에이전트는 자기 행동이 결과에 얼마나 기여했는지 알기 어렵다. 다른 팀원이 성과를 내는 동안 아무것도 하지 않는 "lazy agent"가 생길 수 있다.
- **부분 관측**: 시야 밖의 팀원과 상대를 모르므로 관측만으로는 상태가 Markov가 아니다.

9.2 Parameter sharing

팀원 모두가 하나의 policy network π_θ를 공유하고, 모든 에이전트의 transition으로 같은 파라미터를 갱신한다 [20]. 데이터 효율이 에이전트 수만큼 좋아지며, 관측이 다르면 같은 네트워크도 다르게 행동한다. 에이전트마다 다른 역할이 필요하면 에이전트 번호(one-hot)를 입력에 넣어 구별할 수 있다 [21]. 주의할 점은, 파라미터를 공유하면 개인 보상을 주는 조건도 "서로 독립인 세 이기적 정책"과 같지 않다는 것이다. 결과는 "파라미터를 공유하는 policy에서의 보상 설계 효과"로 해석해야 한다.

9.3 IPPO, MAPPO와 CTDE

- **IPPO**: 각 에이전트가 자기 관측 o^i만 보는 policy와 value function $V(o^i)$로 PPO를 수행한다. 팀원을 환경의 일부로 취급하는 단순한 방법이지만, StarCraft 협력 과제(SMAC)에서 강한 성능을 보였다 [22].
- **MAPPO**: 정책은 그대로 o^i만 보지만, 학습 때만 쓰는 critic은 전체 상태 s를 입력으로 받는다 [23]. 이렇게 *학습은 중앙에서, 실행은 분산해서* 하는 방식을 CTDE(centralized training with decentralized execution)라 한다 [24].

MAPPO 논문은 성능에 영향을 준 요인으로 value 정규화, critic 입력 구성, epoch와 minibatch 수, clipping 계수, 사망 에이전트 처리(death masking)를 꼽는다 [23]. Death masking은 사망한 에이전트의 critic 입력을 에이전트 번호만 남긴 상수 벡터로 바꾸는 방법으로, 부활 대기가 있는 이 게임에서도 고려할 만하다.

실험 설계 관점에서는 critic 입력이 실험 조건마다 달라지면 안 된다. 어떤 조건에서만 centralized critic을 쓰면, 결과 차이가 정보 구조 때문인지 critic 때문인지 구분할 수 없다.

9.4 Credit assignment

COMA [25]는 centralized critic $Q(s, a)$를 학습하고, 에이전트 i의 행동만 바꿔 본 반사실(counterfactual) baseline으로 advantage를 만든다. τ^i는 에이전트 i의 관측 · 행동 이력이다.

$$A^i(s, a) = Q(s, a) - \sum_{a'^i} \pi^i(a'^i \mid \tau^i)\, Q\big(s, (a^{-i}, a'^i)\big). \tag{26}$$

팀원의 행동 a^{-i}는 그대로 두고 자기 행동만 평균적인 선택으로 바꿨을 때보다 얼마나 나았는지를 측정하므로, 팀 보상 하나에서 각자의 기여를 분리하려는 시도다. 가치 기반 방법에서는 팀 가치 Q_{tot}를 에이전트별 가치의 합(VDN [26])이나 단조 결합(QMIX [27])으로 분해하는 접근이 있다. 이 프로젝트의 기본 학습기는 PPO이므로, 이들은 팀 보상의 credit assignment 문제가 실제로 관찰될 때 참고할 대안이다.

9.5 순차 결정

- **HAPPO/HATRPO** [28]: 다중 에이전트 advantage 분해 정리

$$A_\pi^{i_{1:m}}\big(s, \boldsymbol{a}^{i_{1:m}}\big) = \sum_{j=1}^{m} A_\pi^{i_j}\big(s, \boldsymbol{a}^{i_{1:j-1}}, a^{i_j}\big) \tag{27}$$

를 이용한다. 여러 에이전트의 공동 행동 advantage를, 앞 순서 에이전트들의 행동이 정해졌을 때 각 에이전트가 더하는 advantage의 합으로 나눈 것이다. 이를 바탕으로 *policy update*를 무작위 순서에 따라 에이전트별로 차례로 수행하며, 파라미터를 공유하지 않는 이질적 에이전트에서도 단조 개선을 보인다. 실행할 때는 모든 에이전트가 동시에 행동한다.

- **Multi-Agent Transformer(MAT)** [29]: 위 분해를 *행동 선택*에 직접 적용한다. 모든 에이전트의 관측을 encoder로 처리하고, decoder가 $a^{i_1}, a^{i_2}, \ldots$를 자기회귀적으로 생성한다. 즉 m번째 에이전트는 앞 에이전트들의 이번 스텝 행동을 보고 결정한다. PPO 형태의 clipped objective로 학습한다.

이 프로젝트의 순차 결정 조건은 실행 시점의 순차성이므로 MAT에 가깝다. 다만 MAT는 모든 관측을 한 모델이 보는 중앙 실행 구조이고, 이 프로젝트는 각 에이전트가 자기 관측과 앞 순번의 정보만 보는 구조라는 차이가 있다.

9.6 Learned Communication

- **RIAL/DIAL** [30]: RIAL은 메시지를 또 하나의 이산 행동으로 보고 강화학습으로 학습한다. DIAL은 중앙 학습 중에 메시지를 실수 값으로 전달해, 받는 쪽의 gradient가 보내는 쪽까지 흐르게 한다. 실행할 때는 메시지를 이산화한다.
- **CommNet** [31]: 에이전트들의 latent state 평균을 연속 메시지로 공유하며, 여러 라운드의 communication을 한 네트워크 안에서 미분 가능하게 학습한다.
- **TarMAC** [32]: 보내는 쪽이 key(signature)를, 받는 쪽이 query를 만들어 attention으로 누구의 메시지를 얼마나 볼지 정한다. 여러 라운드 통신을 지원한다.
- **Intention Sharing** [33]: 각 에이전트가 self policy와 근사 환경 모델로 앞으로의 짧은 궤적(의도)을 상상해 공유하고, attention으로 중요한 부분을 골라 쓴다. 이 프로젝트의 의도 공유 조건과 가장 가깝다.

Communication을 학습했다고 해서 메시지가 실제로 쓰였다는 보장은 없다 [34]. 메시지가 보내는 쪽의 행동이나 관측과 상관이 있는지(positive signaling)와, 받는 쪽의 행동을 실제로 바꾸는지(positive listening)를 따로 확인해야 한다. 가장 간단한 검증은 학습 후 메시지를 지우거나 다른 에피소드의 메시지로 바꿨을 때 성능과 행동이 달라지는지 보는 것이다. 또한 메시지에 구속력이 없고 정직하게 보낼 유인이 없으면 메시지가 무의미해질 수 있다(cheap talk). 팀이 보상을 공유할수록 정직한 신호를 보낼 유인이 크다.

10 Reward 설계: shaping과 시간 제한

10.1 Potential-based reward shaping

원래 보상 r에 상태의 함수 Φ로 만든 항

$$F(s, s') = \gamma \Phi(s') - \Phi(s) \tag{28}$$

를 더해도 최적 정책은 바뀌지 않는다 [35]. 이 형태가 아닌 shaping은 최적 정책을 바꿀 수 있다. 직관적으로는 할인된 합이 telescoping으로 상쇄되기 때문이다.

$$\sum_{t=0}^{T-1} \gamma^t F(s_t, s_{t+1}) = \gamma^T \Phi(s_T) - \Phi(s_0). \tag{29}$$

중간 경로는 사라지고 처음과 마지막 상태만 남는다. 따라서 다음 두 경우를 구별해야 한다.

- 종료 상태에서 $\Phi(s_T) = 0$으로 두면 shaping은 return에 상수 $-\Phi(s_0)$만 더하므로 원래 목표가 그대로 보존된다.
- $\Phi(s_T)$를 그대로 두면 목표에 마지막 상태의 값 $\gamma^T\Phi(s_T)$가 더해진다. 여전히 마지막 상태로만 정해지는 목표이지만, 원래 목표와 같지는 않다.

식 안의 γ는 학습에 쓰는 discounting ratio와 같아야 한다. γ를 빼고 $\Phi(s') - \Phi(s)$만 쓰면 $\gamma < 1$에서 합이 깔끔하게 상쇄되지 않는다.

Shaping 항은 매 스텝 바로 들어오므로, bootstrapping을 쓰는 학습(GAE의 $\lambda < 1$, TD 학습)에서 즉각적인 신호가 된다. 반대로 $\lambda = 1$인 Monte Carlo return에서는 t 이후 shaping 항의 합이 $\gamma^{T-t}\Phi(s_T) - \Phi(s_t)$로 정리된다. $\Phi(s_T) = 0$이면 이것은 상태만의 함수이므로 baseline을 바꾸는 효과에 가깝다. Potential-based shaping이 Q 함수를 Φ로 초기화하는 것과 동등하다는 결과 [36]도 같은 맥락이다. 다중 에이전트 일반합 게임에서도 potential-based shaping은 Nash equilibrium의 집합을 바꾸지 않는다 [37].

10.2 Territory 보상과 변화량 보상

영역처럼 "얼마나 갖고 있는가"를 나타내는 양 $\Phi(s)$가 있을 때, 매 틱 보상을 $\Phi(s_t)$ 자체(점유한 땅의 넓이)로 줄 수도 있고 $\gamma\Phi(s_{t+1}) - \Phi(s_t)$(변화량)로 줄 수도 있다. 둘은 같은 정보를 쓰지만 목표가 다르다. 보유량 보상의 return은 $\sum_t \gamma^t\Phi(s_t)$이므로 "에피소드 내내 많이 보유하기"를 최적화한다. 예를 들어 중반에 잠시 영역을 내주고 마지막에 크게 되찾는 전략은 손해로 학습된다. 변화량 보상은 위의 telescoping 때문에 마지막 상태만 남는다. 따라서 승패를 마지막 시점에 판정한다면 변화량 형태가 승리 목표와 일치하고, 보유량 형태는 승리 규칙을 누적 점유율 기준(오버워치 점령전과 같은 방식)으로 바꿀 때 일치한다.

10.3 시간 제한

§2에서 정리했듯이, 승패가 정해지는 종료 틱은 termination으로, 학습 편의상 자른 rollout은 truncation으로 처리한다 [38]. 시간 제한이 과제의 일부라면 남은 시간을 관측에 넣는다.

11 프로젝트와의 관계

11.1 환경 (2026.10.31 마지막 수정)

저장소의 `env.md`가 정의하는 게임은 다음과 같다. 세부 규칙과 아직 정하지 않은 규칙은 저장소 문서를 따른다.

- $n \times m$ 타일맵(구현 기본값 12×16)을 반으로 나누어 두 팀이 시작한다. 팀의 영역은 소유한 타일 수다.
- 한 틱에 각 에이전트는 이동 4방향, 인접 타일 공격 4방향, 정지 중 하나를 고른다. 공격한 다음 틱에는 공격할 수 없다.
- 같은 타일을 여러 에이전트가 공격하면 팀별 공격 수가 많은 쪽으로 소유권이 바뀌고, 같으면 유지된다.
- 자기 팀 영역이 아닌 칸에 서 있으면 사망하고, 일정 시간 뒤(구현 기본값 2틱) 자기 영역 안에서 부활한다.
- 종료 틱 T(구현 기본값 256)에 영역이 넓은 팀이 이긴다.

11.2 실험 조건

정보 구조 3가지와 보상 목표 2가지를 조합한 6개 조건을 비교한다.

	T: 팀 승리 우선	**S: 개인 생존 우선**
A. 독립 결정	A-T	A-S
B. 순차 결정	B-T	B-S
C. 의도 공유	C-T	C-S

정보 구조 축은 다음과 같다.

- **A. 독립 결정**: 세 에이전트가 같은 시점의 관측만 보고 동시에 다음 행동을 정한다.
- **B. 순차 결정**: 에이전트에 1, 2, 3 순서를 정한다. 1은 독자적으로 행동하고, 2는 1의 행동과 그에 따른 다음 상태를 보고, 3은 1과 2의 행동 이후 변화된 상태를 보고 행동을 정한다.
- **C. 의도 공유**: 각 에이전트가 행동하기 전에 다음 행동을 공유 메모리에 먼저 제시한다. 제시된 의도를 바탕으로 한 feedback loop로 각 에이전트에 학습 신호를 주고, 이를 바탕으로 최종 행동을 정한다.

보상 축은 다음과 같다.

- **T: 팀 승리 우선**: 모든 조건의 바탕이 되는 팀 보상(§11.3)만 준다.
- **S: 개인 생존 우선**: 같은 팀 보상에, 에이전트 자신이 사망할 때의 패널티를 더한다.

S에서도 팀 승리 보상을 유지하는 이유는, 생존만 보상하면 전선에서 떨어진 내륙에 머무르는 정책이 거의 최적이 되기 때문이다. 이 게임에서는 적이 자기 영역에 인접한 칸만 공격할 수 있으므로, 전선에서 한 칸만 떨어져도 공격받지 않는다.

11.3 보상 명세 (2026.10.31 마지막 수정)

팀 k의 potential을 두 팀 영역 차이의 비율로 둔다. $A_k(s)$는 팀 k의 타일 수, $M = nm$은 전체 타일 수다.

$$\Phi_k(s) = \frac{A_k(s) - A_{-k}(s)}{M} \in [-1, 1], \qquad \Phi_k(s_0) = 0. \tag{30}$$

팀 k의 에이전트 i가 틱 t에서 $t+1$로 넘어갈 때 받는 보상은 다음과 같다.

$$r_t^{\mathrm{T}} = \beta\big(\gamma\Phi_k(s_{t+1}) - \Phi_k(s_t)\big) + \mathbb{I}\{t+1 = T\}\, z_k, \tag{31}$$

$$r_t^{\mathrm{S},i} = r_t^{\mathrm{T}} - c\,\mathbb{I}\{\text{에이전트 } i\text{가 틱 } t+1\text{에 사망}\}. \tag{32}$$

$z_k \in \{+1, 0, -1\}$는 마지막 틱의 승 · 무 · 패다. 팀 보상 r_t^{T}는 팀원 3명에게 같은 값으로 주고(3으로 나누지 않는다), 부활을 기다리는 에이전트에게도 준다. 사망 패널티는 사망한 에이전트 자신만 받는다. 설계 근거와 권고는 다음과 같다.

- **종료 상태의 Φ [미정]**: $\Phi(s_T)$를 0으로 두지 않으면(§10) 학습 목표는 "승패 + $\beta\gamma\times$ 최종 영역 차이"가 된다. $|\Phi| \le 1$이므로 $\beta \le 1$이면 이김 > 비김 > 짐 순서가 유지된다. 최종 영역 차이는 평가 지표인 net territory share와 같고, 상대에게 계속 지는 학습 초반에도 덜 지는 법을 배울 수 있으므로 이 방식을 권고한다.
- **사망 패널티 c [미정]**: 타일 하나가 바뀌면 Φ는 $2/M$만큼 변한다(12×16 맵에서 약 0.0104). $c = \beta \cdot (2/M) \cdot k$로 두면 "한 번 죽는 비용 = 타일 k개"로 해석할 수 있다. 승리 보상 1은 타일 $M/(2\beta)$개(12×16 맵, $\beta = 1$이면 96개) 가치이므로, 에피소드당 예상 사망 횟수 $\times\, k$가 이에 가까워지면 생존이 승리보다 중요해진다. k는 pilot으로 정한다.

- **γ 와 GAE [미정]**: 식 안의 γ는 PPO 학습의 γ와 같은 값을 쓴다. Shaping이 매 틱 신호를 주므로 $\gamma = 0.99$, $\lambda = 0.95$로 시작할 수 있다. 다만 승패 z_k는 256틱 끝에 한 번 들어오므로 $\gamma = 0.995$, 0.999도 비교한다.
- **남은 시간**: 같은 영역 차이라도 남은 시간에 따라 가치가 다르므로 남은 틱 수를 관측에 포함한다. 현재 저장소 구현의 관측에는 이미 포함되어 있다.

11.4 학습 알고리즘의 기본 선택 (2026.10.31 마지막 수정)

- 기본 학습기는 parameter-sharing PPO(IPPO)다. MAPPO의 centralized critic은 확장 후보이며, 쓴다면 6개 조건 모두에 같은 critic 입력을 쓴다.
- 조건 간 차이는 *정책의 입력과 구조*에만 둔다. PPO 업데이트 규칙, 하이퍼파라미터, 보상 계수, 학습 예산, 상대 분포, 평가 seed는 6개 조건에서 같게 둔다.
- B에서는 순번이 다른 에이전트가 같은 네트워크를 쓰므로 순번을 입력에 넣는다(§9의 parameter sharing).
- 공격 cooldown과 맵 경계 때문에 선택할 수 없는 행동은 action mask로 처리한다(§4).

11.5 확정 전에 정할 사항 (2026.10.31 마지막 수정)

- **A의 관측 범위** : A에 팀원의 위치와 직전 행동을 포함하면, B와 C는 "이번 틱의 행동 · 의도 정보"만 추가로 얻어 조건 차이가 깔끔해진다. A를 자기 주변 관측만으로 두면 B와 C는 팀원 상태 정보까지 함께 얻게 되어 두 요인이 섞인다.
- **B의 "다음 상태" [미정]**: 이 게임은 공격 수 비교와 이동 충돌을 6명 동시 판정으로 처리하므로, 1의 행동에 따른 다음 상태는 상대 행동까지 알아야 정해진다. (i) 한 틱 안에서 실제로 순차 실행하면 동시 판정 규칙이 바뀌어, A와의 차이가 정보 때문인지 규칙 때문인지 분리할 수 없다. (ii) 동시 판정을 유지하고 앞 순번의 이번 틱 행동만 제공하면(MAT와 같은 구조) 규칙은 같게 유지된다. 순서를 고정할지 에피소드마다 섞을지도 정해야 한다.
- **C의 메시지와 feedback loop [미정]**: 메시지 내용(다음 행동 하나, 행동 분포, 목표 칸), 구속력(제안과 다른 행동을 할 수 있는지), 통신 라운드 수, 그리고 "학습 신호"가 무엇인지(메시지를 통한 gradient 전달, 메시지를 행동처럼 PPO로 학습, 제안과 실행의 일치에 대한 추가 보상)를 정해야 한다. 추가 보상을 쓰면 C 조건에만 보상 항이 하나 더 생겨 보상 축의 통제가 깨진다. B와의 차이는, B는 앞 순번의 *실제* 행동을 보고 C는 동시에 낸 *제안* 행동을 본다는 점이다.
- **정보 공유 메트릭 [미정]**: 에이전트간 정보 공유는 크게 두 가지로 나눈다. Cross-attention-like method를 통해 각 모델의 parameter를 섞어주는 방식, 모델 끝에 head를 달아 feature를 추출한 뒤 섞어주는 방식, etc.
- **보상 계수 [미정]**: β, c(또는 k), γ, λ. Pilot 후 고정한다.

11.6 문헌에서 가져올 교훈

- **PPO와 구현 세부** (§4): 기본 학습기다. Advantage 정규화와 action masking은 6개 조건 모두에 같게 적용한다.
- **GAE와 DeepSeek-R1 A.3** (§2, §6): 승패는 종료 시점 보상이다. A.3에서 PPO의 λ 조정이 GRPO와의 격차를 줄였듯이, γ와 λ는 결과를 바꿀 수 있는 통제 변수로 기록한다.

- **DAPO dynamic sampling** (§7): 모든 응답이 오답이면 학습 신호가 사라지는 것처럼, 상대에게 전패하면 승패 신호가 사라진다. 상대의 강도를 단계적으로 올려 승률이 0과 1 사이에 머물게 한다.
- **Clip-Higher와 entropy** (§7, §5): 보상이 희소할수록 정책이 일찍 굳지 않게 탐색을 유지하는 장치가 중요하다. PPO에서는 entropy 계수가 이 역할을 한다.
- **TRPO** (§3): PPO의 clipping이 어떤 안정성 문제를 근사하는지 이해하는 이론적 배경이다.
- **SAC, GRPO, SDPO**: 직접 사용하지 않는다. SAC는 off-policy · entropy 기반 탐색의 대조 사례이고, GRPO의 group은 팀원 집합이 아니며, SDPO는 텍스트 피드백을 전제로 한다.
- **다중 에이전트 문헌** (§9): parameter sharing과 IPPO/MAPPO는 기본 구조, COMA는 팀 보상의 credit assignment 문제가 관찰될 때의 대안, MAT와 HAPPO는 B, 통신 문헌은 C의 설계 근거다. C는 학습 후 메시지 제거 실험으로 통신이 실제로 쓰였는지 확인한다 [34].
- **Potential-based shaping** (§10): 승리 목표를 유지하면서 매 틱 신호를 주는 근거다.

11.7 평가 관점

주 지표는 팀 승률과 최종 영역 차이(net territory share)다. 보조 지표로 에이전트별 사망 횟수와 생존 시간, 점령과 방어 사건, 이동 실패, 팀원 간 공격 중복을 기록한다. C에서는 메시지를 제거하거나 섞었을 때의 성능 변화도 기록한다. 상대 봇 유형, 학습 step, random seed를 통제하고, 학습에 쓰지 않은 상대와 좌우를 바꾼 맵에서도 평가한다. Proposal 초안은 최종 평가 상대로 사람 3명을 계획했으며, 유지 여부는 [**미정**]이다. 이들은 *알고리즘 문헌에서 검증된 성과가 아니라 프로젝트의 실험 계획*에 해당한다.

12 종합 비교

방법	데이터	가치함수	학습 신호	프로젝트에서의 위치
TRPO	On-policy	V	GAE advantage, 평균 KL 제약	PPO의 이론적 배경
PPO	On-policy	V	GAE advantage, clipped surrogate	기본 학습기
SAC	Off-policy	두 개의 Q	Soft Q target, entropy 최대화	대조 사례; Spinning Up 구현은 연속 행동용
GRPO	On-policy	없음	같은 입력의 출력 group 안에서 보상 정규화	직접 사용 안 함; group은 팀원 집합이 아님
DAPO	On-policy	없음	GRPO + clip-higher, dynamic sampling, token-level loss, overlong shaping	희소 보상 · 탐색 문제의 참고 사례
SDPO	On-policy	없음	피드백 조건부 self-teacher의 토큰 분포 distillation	선택적 읽기 자료
IPPO / MAPPO	On-policy	$V(o^i)$ / $V(s)$	PPO, parameter sharing, CTDE	기본 구조와 critic 선택
HAPPO / MAT	On-policy	중앙 V	순차 advantage 분해, 자기회귀 행동 선택	조건 B의 근거
DIAL, TarMAC, Intention Sharing	방법별로 다름	방법별로 다름	학습된 메시지(RL 또는 미분 가능한 채널)	조건 C의 근거

13 추가로 읽을 문헌과 우선순위

1. **실험 시작 전 필수**: PPO 원 논문 [9], GAE [10], PPO 구현 세부 [11], invalid action masking [13], MAPPO [23], potential-based shaping [35].
2. **조건 B · C 구현 전**: MAT [29], HAPPO [28], RIAL/DIAL [30], TarMAC [32], Intention Sharing [33], 통신 측정의 함정 [34].
3. **결과 해석과 문제 해결 시**: IPPO [22], parameter sharing [20, 21], COMA [25], VDN/QMIX [26, 27], on-policy 구현 선택의 대규모 비교 [12], 시간 제한 처리 [38], 다중 에이전트 shaping [37], shaping과 Q 초기화의 관계 [36].
4. **지정 자료의 배경**: TRPO 원 논문 [8], SAC 원 논문과 후속 연구 [14, 15, 16], DeepSeekMath [17], Dr. GRPO [18], KL 근사 [19], CTDE의 대표 사례 MADDPG [24].

참고 문헌 및 직접 링크

[1] OpenAI. *Spinning Up: Trust Region Policy Optimization.* `https://spinningup.openai.com/en/latest/algorithms/trpo.html`.

[2] OpenAI. *Spinning Up: Proximal Policy Optimization.* `https://spinningup.openai.com/en/latest/algorithms/ppo.html`.

[3] OpenAI. *Spinning Up: Soft Actor-Critic.* `https://spinningup.openai.com/en/latest/algorithms/sac.html`.

[4] DeepSeek-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.* arXiv:2501.12948. `https://arxiv.org/pdf/2501.12948`. Section 2 및 Appendix A.3 참조.

[5] Q. Yu et al. *DAPO: An Open-Source LLM Reinforcement Learning System at Scale.* arXiv:2503.14476, 2025. `https://dapo-sia.github.io/`; `https://arxiv.org/pdf/2503.14476`.

[6] J. Hübotter et al. *Reinforcement Learning via Self-Distillation.* arXiv:2601.20802. `https://self-distillation.github.io/SDPO`; `https://arxiv.org/abs/2601.20802`.

[7] CS30707 Team 1. *Project Repository.* `https://github.com/a11cho/2026F_CS30707_Team_1`.

[8] J. Schulman et al. *Trust Region Policy Optimization.* ICML 2015. arXiv:1502.05477.

[9] J. Schulman et al. *Proximal Policy Optimization Algorithms.* arXiv:1707.06347, 2017.

[10] J. Schulman et al. *High-Dimensional Continuous Control Using Generalized Advantage Estimation.* ICLR 2016. arXiv:1506.02438.

[11] S. Huang et al. *The 37 Implementation Details of Proximal Policy Optimization.* ICLR Blog Track, 2022. `https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/`.

[12] M. Andrychowicz et al. *What Matters in On-Policy Reinforcement Learning? A Large-Scale Empirical Study.* ICLR 2021. arXiv:2006.05990.

[13] S. Huang, S. Ontañón. *A Closer Look at Invalid Action Masking in Policy Gradient Algorithms.* FLAIRS 2022. arXiv:2006.14171.

[14] T. Haarnoja et al. *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. ICML 2018. arXiv:1801.01290.

[15] T. Haarnoja et al. *Soft Actor-Critic Algorithms and Applications*. arXiv:1812.05905, 2018.

[16] P. Christodoulou. *Soft Actor-Critic for Discrete Action Settings*. arXiv:1910.07207, 2019.

[17] Z. Shao et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv:2402.03300, 2024.

[18] Z. Liu et al. *Understanding R1-Zero-Like Training: A Critical Perspective*. arXiv:2503.20783, 2025.

[19] J. Schulman. *Approximating KL Divergence*. Blog post, 2020. `http://joschu.net/blog/kl-approx.html`.

[20] J. K. Gupta, M. Egorov, M. Kochenderfer. *Cooperative Multi-Agent Control Using Deep Reinforcement Learning*. AAMAS Workshops, 2017.

[21] J. K. Terry et al. *Revisiting Parameter Sharing in Multi-Agent Deep Reinforcement Learning*. arXiv:2005.13625, 2020.

[22] C. Schroeder de Witt et al. *Is Independent Learning All You Need in the StarCraft Multi-Agent Challenge?* arXiv:2011.09533, 2020.

[23] C. Yu et al. *The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games*. NeurIPS 2022 Datasets and Benchmarks. arXiv:2103.01955.

[24] R. Lowe et al. *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments*. NeurIPS 2017. arXiv:1706.02275.

[25] J. Foerster et al. *Counterfactual Multi-Agent Policy Gradients*. AAAI 2018. arXiv:1705.08926.

[26] P. Sunehag et al. *Value-Decomposition Networks for Cooperative Multi-Agent Learning*. AAMAS 2018. arXiv:1706.05296.

[27] T. Rashid et al. *QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning*. ICML 2018. arXiv:1803.11485.

[28] J. G. Kuba et al. *Trust Region Policy Optimisation in Multi-Agent Reinforcement Learning*. ICLR 2022. arXiv:2109.11251.

[29] M. Wen et al. *Multi-Agent Reinforcement Learning is a Sequence Modeling Problem*. NeurIPS 2022. arXiv:2205.14953.

[30] J. Foerster et al. *Learning to Communicate with Deep Multi-Agent Reinforcement Learning*. NeurIPS 2016. arXiv:1605.06676.

[31] S. Sukhbaatar, A. Szlam, R. Fergus. *Learning Multiagent Communication with Backpropagation*. NeurIPS 2016. arXiv:1605.07736.

[32] A. Das et al. *TarMAC: Targeted Multi-Agent Communication*. ICML 2019. arXiv:1810.11187.

[33] W. Kim, J. Park, Y. Sung. *Communication in Multi-Agent Reinforcement Learning: Intention Sharing*. ICLR 2021.

[34] R. Lowe et al. *On the Pitfalls of Measuring Emergent Communication*. AAMAS 2019. arXiv:1903.05168.

[35] A. Y. Ng, D. Harada, S. Russell. *Policy Invariance under Reward Transformations: Theory and Application to Reward Shaping*. ICML 1999.

[36] E. Wiewiora. *Potential-Based Shaping and Q-Value Initialization are Equivalent*. Journal of Artificial Intelligence Research 19, 2003.

[37] S. Devlin, D. Kudenko. *Theoretical Considerations of Potential-Based Reward Shaping for Multi-Agent Systems*. AAMAS 2011.

[38] F. Pardo et al. *Time Limits in Reinforcement Learning*. ICML 2018. arXiv:1712.00378.